

We start by issuing the `makemigrations` command ❶. In the output, Django indicates that we're trying to add a required (*non-nullable*) field to an existing model (`topic`) with no default value specified ❷. Django gives us two options: we can provide a default right now, or we can quit and add a default value in `models.py` ❸. Here I've chosen the first option ❹. Django then asks us to enter the default value ❺.

To associate all existing topics with the original admin user, `ll_admin`, I entered the user ID of 1 ❻. You can use the ID of any user you've created; it doesn't have to be a superuser. Django then migrates the database using this value and generates the migration file `0003_topic_owner.py`, which adds the field `owner` to the `Topic` model.

Now we can execute the migration. Enter the following in an active virtual environment:

```
(ll_env)learning_log$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, learning_logs, sessions
Running migrations:
❶ Applying learning_logs.0003_topic_owner... OK
(ll_env)learning_log$
```

Django applies the new migration, and the result is OK ❶.

We can verify that the migration worked as expected in a shell session, like this:

```
>>> from learning_logs.models import Topic
>>> for topic in Topic.objects.all():
...     print(topic, topic.owner)
...
Chess ll_admin
Rock Climbing ll_admin
>>>
```

We import `Topic` from `learning_logs.models` and then loop through all existing topics, printing each topic and the user it belongs to. You can see that each topic now belongs to the user `ll_admin`. (If you get an error when you run this code, try exiting the shell and starting a new shell.)

NOTE

You can simply reset the database instead of migrating, but that will lose all existing data. It's good practice to learn how to migrate a database while maintaining the integrity of users' data. If you do want to start with a fresh database, issue the command `python manage.py flush` to rebuild the database structure. You'll have to create a new superuser, and all of your data will be gone.

Restricting Topics Access to Appropriate Users

Currently, if you're logged in, you'll be able to see all the topics, no matter which user you're logged in as. We'll change that by showing users only the topics that belong to them.